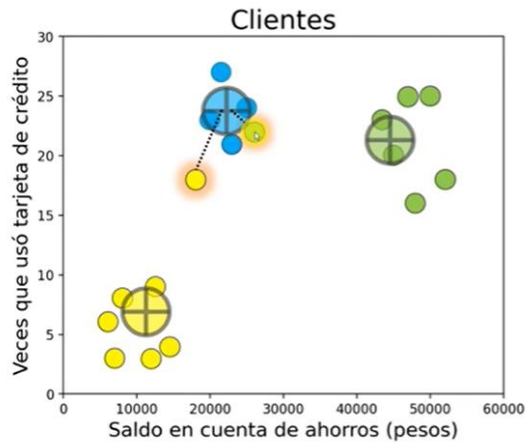


KMEANS

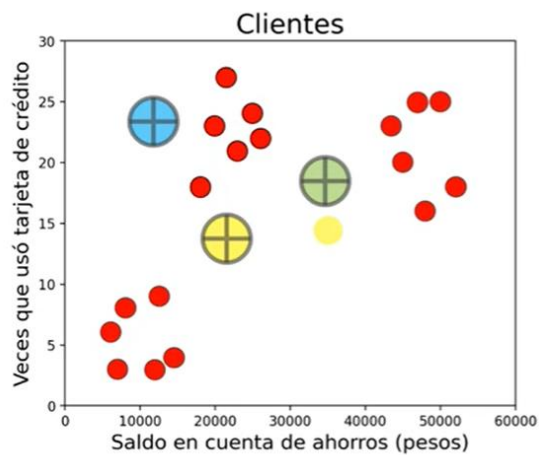
1. Se forman **centroides** aleatorios de numero k.



2. Cada dato se asigna al centroide **más cercano**.



3. Se calcula el promedio de cada uno y se mueve el centroide a esa ubicación.



4. Repetir hasta converger.

¿Calcular K ideal?

Elbow Method

➔ Con la métrica **inercia**.

La **inercia** mide que tan lejos están los puntos de sus respectivos centroides.

¿Qué hace?

1. Ejecuta **kmeans** varias veces, probando con $k=1, k=2, k=3, \dots$
2. Grafica los resultados: eje x la k (clusters) y en el eje y la inercia.
3. La curva baja drásticamente y luego se aplana. Donde la curva parece un codo es la k ideal

```
from matplotlib import markers
from sklearn.cluster import KMeans

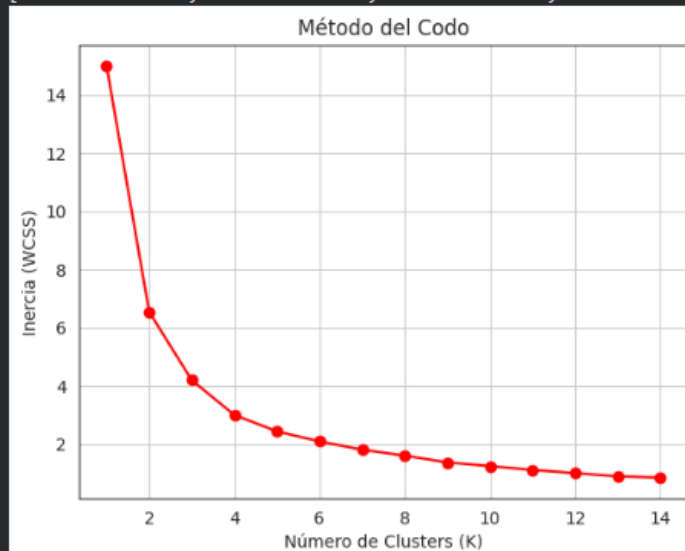
#Elbow method

wcss=[]

for i in range(1,15):
    kmeans=KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(x_scaler)
    wcss.append(kmeans.inertia_)

print('Inercias: ')
print(wcss)
plt.plot(range(1,15),wcss,marker='o',color='red')
plt.title('Método del Codo')
plt.xlabel('Número de Clusters (K)')
plt.ylabel('Inercia (WCSS)')
plt.grid(True)
plt.show()
```

... Inercias:
[15.004048209430135, 6.544280052364503, 4.194331975998724, 2.997493245903521, 2.43...



Integración con otros Modelos (KNN y Random Forest)

Se pasan nuestros datos calculados en k-means

```
kmeans_ideal= KMeans(n_clusters=4,init='k-means++', random_state=42, n_init=10)
clusters_asignados=kmeans_ideal.fit_predict(x_scaler)
```

K-Means no solo sirve para agrupar, sino que es una herramienta poderosa de Ingeniería de Variables (Feature Engineering) para potenciar modelos supervisados:

Random Forest: Podemos usar K-Means para crear una nueva columna (feature) con el ID del cluster. Esto ayuda al Random Forest a detectar patrones espaciales complejos que quizás no vería solo con las variables originales, mejorando su precisión.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
import numpy as np

X=df_random_forest
y=wine.target

X_train,X_test, y_train, y_test=train_test_split(X, y, test_size=0.3, random_state=42)

#Creacion y entrenamiento random forest
rf_model=RandomForestClassifier(n_estimators=100,random_state=42)
rf_model.fit(X_train,y_train)

#Predicciones
pred=rf_model.predict(X_test)
print(f"Precisión del Random Forest con feature de clustering: {accuracy_score(y_test, pred):.2%}")

*** Precisión del Random Forest con feature de clustering: 98.15%
```

KNN (K-Nearest Neighbors): En datasets masivos, KNN es muy lento. Podemos usar K-Means para reducir los datos a solo 'k centroides'. Luego, KNN clasifica los puntos nuevos comparándolos con estos centroides representativos en lugar de con millones de puntos, acelerando enormemente el tiempo de predicción.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn_model=KNeighborsClassifier(n_neighbors=4)
knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)
knn_acc = accuracy_score(y_test, knn_pred)

rf_acc = accuracy_score(y_test, pred)
print(f"Precisión Random Forest: {rf_acc:.4f}")
print(f"Precisión KNN: {knn_acc:.4f}")

Precisión Random Forest: 0.9815
Precisión KNN: 0.7222
```